

Software Transactional Memory

STM lets you write concurrent updates as if they were ordinary sequential code.

Outline

1. The Problem
2. The STM Concept
3. What makes it work?
4. The Cost & Tradeoffs
5. What's interesting about OCaml 5?
6. The Demo

1. The Problem

Main point

The primary issue is that standard lock-based concurrency forces the programmer to manage manual synchronization, which is difficult to compose and highly error-prone.

Supporting points

- **Race conditions:** When threads interleave, they can observe partially updated state, leading to inconsistent results and non-deterministic behavior.
- **Composability failure:** It is difficult to combine two thread-safe operations into one atomic unit without designing a new, typically more complex locking protocol.
- **Deadlocks/Complexity:** Protecting shared state with locks requires careful ordering, which often leads to deadlocks and lock-ordering as program complexity grows.

Example

Thread A:	Thread B:
read x	read x
x = x + 1	x = x + 1
write x	write x
Expected: 2	Actual: 1

2. The STM Concept

Promise

Write code as a transaction, and the system will make it atomic.

Key ideas

Speculative. A transaction acts like it has a private, consistent view of memory, allowing it to perform computations without blocking others.

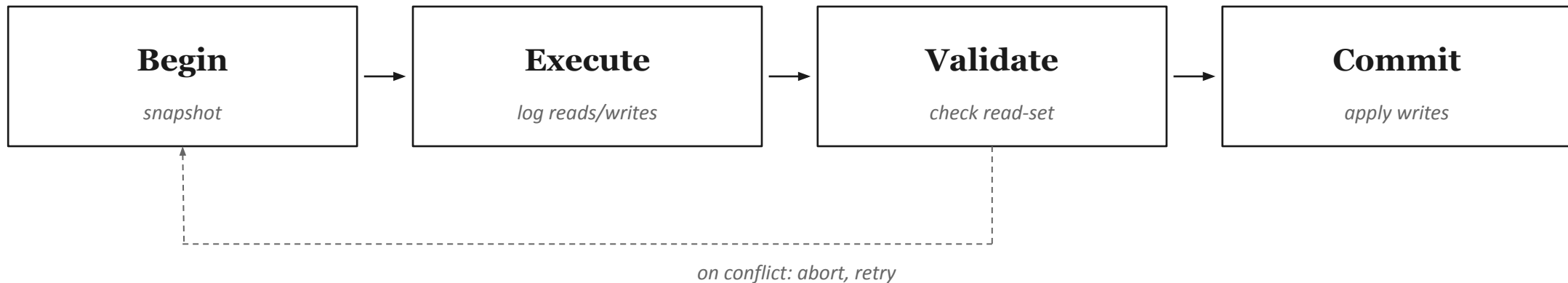
Tracked. The runtime silently logs all shared variables that the transaction reads or writes, creating a footprint of the operation.

Validated. Before final commitment, the system checks whether that footprint is still valid; if the underlying data changed, the transaction realizes its work was based on stale information and restarts.

3. What makes it work?

STM is a validation system on optimistic concurrency.

The Transaction Lifecycle



Optimistic. The transaction executes as if it had exclusive access, keeping tentative changes in a private write-set. The programmer only deals with a collision, if a collision actually happens, which is much cheaper than blocking all threads with locks by default.

Read/write log. The runtime captures every read and write inside a local, thread-specific structure, typically a hash table. The read-set remembers the version of data the thread saw, and the write-set holds the threads pending updates. This creates a private “scratchpad” illusion essential for transactional isolation.

Validation. At the Commit stage, the system performs a linear scan or version check. It essentially asks: “*Is the version I read still the current version in memory?*”. If the answer is yes for everything in the read-set, the transaction commits and writes get published atomically. If any location has changes, the validation aborts, the write-set is cleared, and the transaction retries.

3. What makes it work? - continued

Mechanism

Why it's safe

- **Isolation:** Changes are buffered privately, so no other thread sees intermediate inconsistent values.
- **Atomicity:** The Commit phase is a single, uninterruptible point where the writes are published.
- **Consistency:** If the validation fails, the transaction is aborted, preventing invalid state from ever reaching shared memory.
- **Linearizability:** The entire transaction appears to take effect at exactly the time of its commit, matching a serial, sequential execution order.

Pseudocode

```
atomically {  
    let r = read  acct_a  
    let s = read  acct_b  
    write acct_a (r - n)  
    write acct_b (s + n)  
}  
  
// runtime, on commit:  
//   1. re-read read-set  
//   2. if changed -> abort, retry  
//   3. else -> publish writes
```

4. The Cost & Tradeoffs

Analogy

STM is to locks what Garbage Collection (GC) is to manual memory management.

Where it works

- **Composability:** Transactions can be easily nested or combined into larger, complex operations without worrying about deadlocks.
- **Developer Productivity:** Programmers write code in a natural, direct style without the manual complexity of “acquire/release” lock protocols.

Where it hurts

- **Wasted work:** Under high contention, the same transaction might run to completion multiple times only to fail at the last second, consuming CPU cycles for nothing.
- **Fairness/Starvation:** If one transaction is always slower than others, it may never succeed, leading to starvation.
- **Side effects in transactions:** If you perform an I/O operation inside a transaction, that action may happen multiple times if the transaction retries.

5. What's interesting about OCaml 5?

Why OCaml 5 + Algebraic Effects

- Effects are a mechanism for delimited continuations, separating the *description* of an effect from its *handler* (the implementation)
- Effects provide a lightweight, non-blocking way to handle control flow. With effects, you don't just call a function; you perform an entire operation.
- The effect handler acts as a transaction coordinator. It can intercept the effects, record them in the log and restart the whole thread from the beginning if a conflict occurs. This makes the retry logic clean.
- Utilizing effects and effect handlers, STM systems can be created/prototyped. Defining a function called `atomically` that acts as a handler that wraps business logic. The usage feels like sequential code, but under the hood, the runtime is doing the heavy lifting of logging, validating, and retrying.

Effect signature

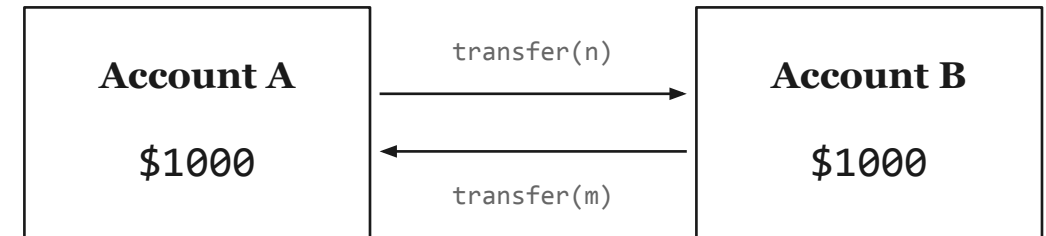
```
type _ Effect.t +=  
  | Read  : 'a tvar -> 'a Effect.t  
  | Write : 'a tvar * 'a -> unit Effect.t  
  
let atomically (f : unit -> 'a) =  
  (* handler installs read/write log *)  
  (* on commit: validate + apply      *)  
  (* on abort:  discard + retry       *)  
  ...
```


6. Demo - two bank accounts

What to observe

- **Invariant preservation:** Despite multiple threads trying to move money, the total balance never fluctuates.
- **Retry visibility:** Logging a message to stdout every time a transaction aborts or retries.
- **Direct style:** Notice there are no locks or try/finally blocks within the business logic. It's just simple sequential code.

Setup



*Invariant: $bal(A) + bal(B) = constant$
where $constant = 2000$*

```
$ dune exec ./bin/demo.exe
```

Summary

- 1.** Locks make concurrency correct by exclusion.
- 2.** STM makes it correct by speculation and validation.
- 3.** OCaml 5 effects let us prototype it cleanly.

Questions?

References

- 1.** Wikipedia, "Software transactional memory," Wikimedia Foundation, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Software_transactional_memory
- 2.** Wikipedia, "Transactional memory," Wikimedia Foundation, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Transactional_memory
- 3.** Wikipedia, "Optimistic concurrency control," Wikimedia Foundation, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Optimistic_concurrency_control
- 4.** B. Berkowitz, "5.3 Concurrency via Software Transactional Memory," in *Concurrent Programming* (online thesis), 2018. [Online]. Available: https://berb.github.io/diploma-thesis/original/o53_stm.html

References - continued

5. OCaml.org, "24 Effect handlers," OCaml 5.4 Manual, 2026. [Online]. Available: <https://ocaml.org/manual/effects.html>

6. L. Bormann, "Playing with OCaml Effects," lewinb.net, 2023. [Online]. Available: https://lewinb.net/posts/17_playing_with_ocaml_effects/